Listing 5.19 Enabling `integration` profile for the integration tests

```
package com.polarbookshop.catalogservice;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@ActiveProfiles("integration")                                          ❶
class CatalogServiceApplicationTests {
  ...
}
```

❶ Enables the “integration” profile to load configuration from application-integration.yml

Open a Terminal window, navigate to the root folder of your Catalog Service project, and run the following command to ensure the tests are successful. Under the hood, Testcontainers will create a PostgreSQL container before the test’s execution and remove it at the end.

```
$ ./gradlew test --tests CatalogServiceApplicationTests
```

Good job! You have added data persistence to a Spring Boot application and written tests while ensuring environment parity. Let’s move on and complete this chapter by discussing how to manage schemas and data in production.

5.4 Managing databases in production with Flyway

It’s good practice to register any database changes, just like you do for your application source code through version control. You’ll need a deterministic and automated way to infer the database’s state, whether specific changes have already been applied, how to recreate a database from scratch, and how to migrate it in a controlled, repeatable, and reliable way. The continuous delivery approach encourages automating as much as possible, including database management.